

PHASE 5 — SECURITY REVIEW

Cyber-Physical Smart Building Access Control System

1. Scope and Method

This is an internal design and code review (P5.5), not a live penetration test — there is no deployed hardware or network to attack in this environment. The review covers the firmware (door_node_firmware/), backend (backend/), and dashboard (dashboard/) as delivered through Phase 5, checked against the threats identified in the original proposal review and the System Design Document's security section.

Where a mitigation has been implemented, this document says so and points at the file/test that proves it. Where it hasn't, that is stated plainly rather than implied away — the goal is an accurate picture for the committee and for whoever picks up Phase 6, not a clean-looking checklist.

2. Threat Model Summary

Asset	Threat actor / scenario	Primary risk
Physical door lock	Someone presents a cloned/emulated card UID	Unauthorized entry
Door node hardware	Someone opens the enclosure to tamper with wiring	Bypass of lock control, sensor spoofing
Credential database	Attacker gains DB read access (backup leak, SQL injection, etc.)	Mass credential/UID exposure
MQTT broker traffic	Network eavesdropper on the door-to-broker link	Credential/event interception
Admin dashboard/API	Credential stuffing or brute force against /api/auth/login	Account takeover
Door node firmware	Physical access to a node, dumping flash	Extraction of the shared AES key, Wi-Fi credentials

3. Before / After — Gaps Flagged Since the Original Proposal Review

Gap flagged	Status	What changed
Card cloning — UID-only read, no cryptographic proof of card identity	Mitigated	desfire_auth.cpp implements DESFire EV1 AES-128 mutual authentication; CARD_SCAN now requires it to pass, not just a UID match
Credential data stored in plaintext at rest	Mitigated	app/crypto.py — Fernet-encrypted card_uid + keyed HMAC blind index for lookup (backend/tests/test_crypto.py)
No login rate limiting / brute-force protection	Mitigated	app/rate_limit.py — per-account lockout after repeated failures (backend/tests/test_auth.py)
Tamper detection only alerted, took no protective action	Mitigated	Tamper interrupt now forces immediate relock and blocks further card entry until admin-cleared via MQTT
No local resilience if MQTT/network drops mid-operation	Mitigated	Offline event ring buffer (queueOfflineEvent/flushOfflineEvents) — events survive a broker outage, RAM-only
Fire/life-safety egress override on fail-secure doors	Mitigated (Phase 1)	REX button + mechanical key override on every door, independent of firmware state; confirmed still functional during

Gap flagged	Status	What changed
		tamper lockout
Per-card key diversification	Not mitigated	All cards at a site share one AES key (DESFIRE_AES_KEY) — see Section 4
MQTT topic/broker access control per device	Partial	TLS + username/password supported in both firmware and backend config; per-device certificate-based auth not implemented
JWT revocation / session management	Not mitigated	Tokens are valid until expiry (60 min default) with no revocation list — logging out client-side doesn't invalidate a leaked token early
Firmware secure boot / code signing	Not mitigated	Out of scope for this project's timeline — flagged as a real gap for a production deployment, not something addressed here

4. Residual Risks and Recommendations

4.1 Shared AES key across all cards

Every door node currently authenticates cards against the same DESFIRE_AES_KEY. If that key is ever extracted from one node's flash (physical access + flash dump), every card at the site is compromised, not just that node's. The standard mitigation is per-card key diversification — deriving each card's key from a master key and the card's UID via AES-CMAC (documented in NXP's application notes) — which was not implemented here due to time/complexity, and is the single highest-value next security investment if this project continues past Phase 5.

4.2 DESFire handshake is unverified against real hardware

The AES mutual-authentication implementation was written against the publicly documented DESFire EV1 protocol and reviewed line-by-line, but the ESP32 toolchain could not be compiled in this environment (no hardware, limited sandboxed network access to the Espressif platform download), so it has never actually run against a real card. Byte-order or framing mistakes in a hand-implemented crypto handshake are exactly the kind of bug that looks correct on paper and fails — or worse, silently accepts something it shouldn't — on real hardware. This must be bench-tested against real DESFire cards before any reliance on it.

4.3 Encryption key management

CREDENTIAL_ENCRYPTION_KEY and CREDENTIAL_INDEX_KEY fall back to a deterministic dev-only key derived from a fixed string if unset, so the project runs out of the box without extra setup. This is safe for development but must never happen in a real deployment — both keys must be set to real random values and kept outside version control, exactly like JWT_SECRET already is. Losing or rotating CREDENTIAL_ENCRYPTION_KEY without a migration plan makes existing encrypted card UIDs permanently unreadable, which is worth a deliberate key-rotation runbook before going further.

4.4 Rate limiting is per-process, in-memory

app/rate_limit.py tracks failed logins per process. A single backend instance (this project's target) is fine; a multi-instance deployment behind a load balancer would need a shared store (Redis, or a DB-backed table) so attempts are tracked consistently across instances.

4.5 Offline event buffer is RAM-only

Buffered access events are lost if a door node loses power while offline (not just network-down) — they exist only in RAM, not flash. Persisting the queue to LittleFS would close this gap if audit completeness during extended outages becomes a requirement.

5. Design/Code Review Checklist

Check	Status	Notes
Passwords hashed, never stored/logged in plaintext	Mitigated	bcrypt via app/security.py, verified in backend tests
Credential secrets (card UIDs) encrypted at rest	Mitigated	app/crypto.py, Fernet + blind index
Transport encryption for MQTT	Mitigated (config-level)	TLS supported in firmware (WiFiClientSecure) and backend (MQTT_USE_TLS); depends on being enabled against a real broker with a real cert, not self-signed/skipped
Role-based access control enforced server-side, not just hidden in UI	Mitigated	require_admin dependency in FastAPI routes; dashboard's UI restrictions are cosmetic on top of this, not the actual boundary
Brute-force protection on authentication	Mitigated	app/rate_limit.py
Tamper response beyond alerting	Mitigated	Forced relock + lockout, Phase 5 firmware
Life-safety egress not blocked by security logic	Mitigated	REX button bypasses tamper lockout and credential checks entirely
Cryptographic (not just UID) card authentication	Mitigated, unverified on hardware	See Section 4.2
Per-credential key diversification	Not mitigated	See Section 4.1
Audit log resilience to network/power loss	Partial	Network loss covered (offline buffer); power loss to a node is not (RAM-only)
Session/token revocation	Not mitigated	See Section 3
Firmware code signing / secure boot	Not mitigated	Out of scope for this project

6. Conclusion

Phase 5 closes the specific gaps flagged earliest and most repeatedly in this project's review history: cloneable card credentials, plaintext data at rest, no brute-force protection, and tamper detection with no teeth. What remains open — key diversification, hardware-verified crypto, session revocation, firmware code signing — are reasonable boundaries for a graduation project's scope, but each is named explicitly here rather than left implicit, so a reader (or a future phase) knows exactly what "security" does and doesn't mean for this system as it stands.